EXPERIMENT 5

Title: Implementation of the Porter Stemming Algorithm

Aim:

To implement the Porter Stemming Algorithm using Python and NLTK to reduce English words to their stems. Stemming is an important preprocessing step in Natural Language Processing that reduces variations of words to a common form. For example, words such as playing and played can be reduced to play, helping NLP systems identify related word forms.

Algorithm:

1. Start the program.
2. Import the PorterStemmer class from the NLTK library.
3. Create an instance of the Porter Stemmer.
4. Define a list containing words with different suffixes.
5. Display a heading indicating the original word and its stem.
6. Read each word from the list.
7. Pass the word to the stem() method of the Porter Stemmer.
8. Apply the algorithm's suffix-removal rules to obtain the stem.
9. Print the original word and the generated stem.
10. Repeat the process until all words have been processed.
11. Stop the program.

Program:

Exp-05-Porter Stemmer Algorithm.py - C:\Users\jerin\OneDrive\Desktop\NLP\Experiments\E...

File Edit Format Run Options Window Help

```
from nltk.stem import PorterStemmer

ps = PorterStemmer()

words = ["playing", "played", "connected",
         "connection", "fairness", "easily"]

print("Porter Stemmer Results")

for word in words:
    print(word, "->", ps.stem(word))
```

Output:

```
>>>
= RESTART: C:\Users\jerin\OneDrive\Desktop\NLP\Experiments'
 Algorithm.py
Porter Stemmer Results
playing -> play
played -> play
connected -> connect
connection -> connect
fairness -> fair
easily -> easili
>>>
```

Result:

Thus, the Porter Stemming Algorithm was implemented successfully using Python and NLTK. The program reduced words with different suffixes to their corresponding stems. This helps reduce vocabulary size and improves text processing efficiency in applications such as document retrieval, text classification, and search engines. The generated stem may not always be a valid dictionary word.